

Sarathi — MVP Document

Version: 2.0 | **Date:** April 2026 | **Status:** Draft — Google Cloud Edition

Table of Contents

- 1. MVP Scope & Goals
- 2. Google Cloud Services Used in MVP
- 3. Features & Acceptance Criteria
- 4. Sprint Plan
- 5. Definition of Done
- 6. Out of Scope for MVP
- 7. Risk Register

1. MVP Scope & Goals

1.1 What MVP Means for Sarathi

The Sarathi MVP is a **single-user personal prototype** that validates the core lifecycle loop end to end. Everything runs on **Google Cloud credits** — one billing account, one platform, zero separate subscriptions.

The goal is to answer one question:

> *Does an AI charioteer that knows your goals, follows your full day, and holds you accountable actually change how you work and live?*

If yes, we scale. If no, we learn and iterate.

1.2 MVP Success Criteria

Metric	Target
Daily active use	Creator uses Sarathi every day for 2 weeks straight
Morning briefing quality	Feels personal and motivating, not robotic
Capture accuracy	80%+ of day captures correctly surfaced in evening debrief
Screen monitoring	Distractions flagged within 30 seconds of threshold breach

Metric	Target
Night debrief	Alarm set through Sarathi at least 10 out of 14 nights
Discipline impact	Creator self-reports measurable improvement in task completion
Cost	All infra costs covered by Google Cloud credits

1.3 MVP Technology Decisions

Decision	Choice	Reason
Cloud LLM	Gemini 2.5 Pro on Vertex AI	Google Cloud credits, no separate billing
Quick tasks LLM	Gemini 2.5 Flash-Lite	Cheap and fast for categorization
Auth	Firebase Authentication	Free, deeply integrated with Android
Database	Cloud SQL (PostgreSQL 15)	Managed, no server ops
Cache	Memorystore Redis	Managed, same GCP project
Vector DB	Vertex AI Matching Engine	Replaces Pinecone, billed to GCP credits
File storage	Cloud Storage (GCS)	Replaces S3, billed to GCP credits
Push notifications	Firebase Cloud Messaging	Free, best Android integration
Voice transcription	Cloud Speech-to-Text (Chirp)	Best quality, GCP credits
Geofencing	Google Maps Platform	Most reliable on Android
Backend hosting	Cloud Run	Serverless, pay per request
Secrets	Secret Manager	Secure, GCP native
CI/CD	Cloud Build	GCP native, free tier generous
Local model (MVP)	Skip — use Gemini API directly	Simplifies MVP, validate concept first

> **Note on local model in MVP:** In MVP, screen monitoring frames are sent to **Gemini 2.5 Flash-Lite** (fast, cheap) instead of a local Ollama model. Local Gemma via Ollama/MediaPipe is a Phase 2 addition once the concept is validated. The privacy trade-off is acceptable for a single-user prototype.

2. Google Cloud Services Used in MVP

2.1 Services Checklist

Google Cloud Service	Used For	Free Tier?
Vertex AI — Gemini 2.5 Pro	Morning briefing, evening/night debrief, main chat	Paid (credits)
Vertex AI — Gemini 2.5 Flash-Lite	Capture categorization, screen classification	Paid (credits, very cheap)
Vertex AI — text-embedding-004	Embedding memories for RAG	Paid (credits)
Vertex AI — Matching Engine	Vector similarity search (RAG retrieval)	Paid (credits)
Cloud Speech-to-Text (Chirp)	Voice capture transcription	60 min/month free
Cloud Text-to-Speech	Sarathi speaking responses (optional)	4M chars/month free
Cloud Run	FastAPI backend hosting	2M requests/month free
Cloud SQL	PostgreSQL database	Paid (credits)
Memorystore	Redis session cache	Paid (credits)
Cloud Storage (GCS)	Image captures, model files	5GB free
Firebase Authentication	User login + JWT	Free (Spark plan)
Firebase Cloud Messaging	Push notifications	Free
Firebase Crashlytics	Android crash reporting	Free
Google Maps Platform	Geofencing + Routes + Places	\$200 free credit/month
Secret Manager	API keys, credentials	First 6 secrets free
Cloud Build	CI/CD pipeline	120 min/day free
Artifact Registry	Docker images	0.5GB free
Cloud Logging	All service logs	50GB/month free
Cloud Monitoring	Dashboards + alerts	Basic free
Cloud Scheduler	Fine-tuning trigger (Phase 3)	3 jobs free

2.2 Setup Order (Before Coding)

Before Sprint 1, complete this GCP setup checklist:

- ☐ Create GCP project: sarathi-prod
- ☐ Enable billing account + link Google Cloud credits

- [] Enable APIs: Vertex AI, Cloud Run, Cloud SQL, Memorystore, Cloud Storage, Speech-to-Text, Text-to-Speech, Maps Platform, Secret Manager, Cloud Build, Artifact Registry
 - [] Create Firebase project + link to GCP project
 - [] Enable Firebase Authentication (Email/Password provider)
 - [] Enable Firebase Cloud Messaging
 - [] Create Cloud SQL instance (PostgreSQL 15, db-f1-micro for MVP)
 - [] Create Memorystore Redis instance (1GB, Basic tier)
 - [] Create GCS bucket: sarathi-app (regional, us-central1)
 - [] Create Vertex AI Matching Engine index
 - [] Add Maps API key to Secret Manager
 - [] Add all other secrets to Secret Manager
 - [] Create service account for Cloud Run with correct IAM roles
 - [] Set up Cloud Build trigger on GitHub main branch
-

3. Features & Acceptance Criteria

FEATURE 01 — User Onboarding & Profile Setup

Description: First-time setup where the user registers via Firebase Auth, sets their goals, daily schedule, and baseline preferences. Data seeds Cloud SQL and Vertex AI Matching Engine.

Google Cloud Services: Firebase Authentication, Cloud Run (FastAPI), Cloud SQL, Vertex AI Matching Engine, text-embedding-004

Acceptance Criteria:

- [] User registers with email + password via Firebase Authentication
 - [] Firebase UID stored in Cloud SQL users table on first login
 - [] User can set up to 10 goals with title, description, and priority
 - [] User can configure daily schedule: wake time, gym time, college leave time, sleep target
 - [] User can set home location (lat/lng via Google Places API autocomplete)
 - [] All setup data persisted to Cloud SQL
 - [] Goals and schedule embedded via text-embedding-004 and stored in Vertex AI Matching Engine
 - [] Onboarding completable in under 5 minutes
 - [] User sees a summary of their profile before finishing setup
-

FEATURE 02 — Morning Wake-Up & Alarm Monitor

Description: Sarathi monitors alarm time. If missed or snoozed, the app proactively opens with a goal-tied motivational prompt generated by Gemini. Firm, no hollow motivation.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Pro, Memorystore Redis, Firebase Cloud Messaging

Acceptance Criteria:

- [] User can set alarm time inside the app (stored in Cloud SQL + sets native Android alarm)
 - [] WorkManager job checks at alarm time if user has dismissed the alarm
 - [] If alarm missed: Android app opens fullscreen wake-up screen automatically via FCM push
 - [] Wake-up message generated by Gemini 2.5 Pro references user's top goals by name
 - [] Tone is firm and direct — no generic motivational quotes
 - [] User must tap "I'm awake" to proceed to morning briefing
 - [] If alarm dismissed on time: app opens briefing screen directly without wake-up screen
 - [] Alarm state tracked in Memorystore Redis to avoid duplicate triggers
-

FEATURE 03 — Morning Briefing

Description: After wake-up, Sarathi delivers a personalized morning briefing. Gemini 2.5 Pro generates it using RAG context from Vertex AI Matching Engine — goals, schedule, pending captures, habit summaries.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Pro, Vertex AI Matching Engine, text-embedding-004, Memorystore Redis, Google Maps Routes API

Acceptance Criteria:

- [] Briefing generated by Gemini 2.5 Pro using top-8 RAG results from Matching Engine
 - [] Briefing includes: top 3 goals for today, workout plan if gym day, breakfast suggestion, time-to-leave for gym and college
 - [] Time-to-leave calculated via Google Maps Routes API using home location + current traffic
 - [] Live countdown timer on screen: updates in real time without page refresh
 - [] User can ask Sarathi follow-up questions in a conversational UI (Gemini streaming)
 - [] Briefing loads within 5 seconds on normal internet
 - [] Briefing feels unique each day — shaped by different RAG context each morning
 - [] Conversation turn stored in Memorystore Redis + Cloud SQL
-

FEATURE 04 — Quick Capture

Description: During the day, user captures tasks, thoughts, or information in three modes: voice (Chirp), text, or image (GCS). Gemini Flash-Lite auto-categorizes each capture.

Google Cloud Services: Cloud Run (FastAPI), Cloud SQL, Cloud Storage (GCS), Cloud Speech-to-Text (Chirp), Vertex AI Gemini 2.5 Flash-Lite, Vertex AI Matching Engine, text-embedding-004

Acceptance Criteria:

- [] Voice capture: tap mic, speak, audio streamed to Cloud Speech-to-Text (Chirp), transcript saved

- [] Text capture: type into field, submit — saved immediately
 - [] Image capture: CameraX photo uploaded to GCS bucket (sarathi-app/user-images/{uid}/), signed URL stored in Cloud SQL
 - [] All captures auto-categorized by Gemini 2.5 Flash-Lite: task / reminder / observation / question
 - [] Capture takes under 10 seconds from open to saved (voice and text modes)
 - [] Capture accessible via persistent floating action button in app
 - [] User can view list of all today's captures sorted by time
 - [] User can mark a capture as resolved
 - [] Each capture embedded via text-embedding-004 and indexed in Vertex AI Matching Engine
 - [] Chirp transcription accuracy acceptable for Hindi and English
-

FEATURE 05 — Homecoming Detection & Trigger

Description: When user arrives home, Google Maps Geofencing API detects it. Firebase Cloud Messaging sends a push notification. User taps to start the evening debrief.

Google Cloud Services: Google Maps Platform (Geofencing API), Firebase Cloud Messaging, Cloud Run (FastAPI)

Acceptance Criteria:

- [] User sets home location in settings using Google Places API autocomplete
 - [] Google Maps Geofencing API creates a 100m radius geofence around home on app start
 - [] Geofence ENTER transition triggers POST to FastAPI /lifecycle/homecoming via FCM data message
 - [] FCM push notification sent to device: "Hey, you're back. Let's talk about your day."
 - [] Tapping notification opens the evening debrief chat screen
 - [] Trigger fires only if current lifecycle phase in Redis is day
 - [] Manual fallback button in Settings to trigger homecoming manually
 - [] Geofence resets each morning so it fires again the next day
 - [] Geofence registration survives app restart (stored in Room DB)
-

FEATURE 06 — Evening Debrief Conversation

Description: Natural language conversation where Sarathi reviews the day, surfaces captures from Cloud SQL, checks goal progress, and plans the evening work session. Powered by Gemini 2.5 Pro with full RAG context.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Pro, Vertex AI Matching Engine, Memorystore Redis, Cloud SQL, Firebase Cloud Messaging

Acceptance Criteria:

- [] Sarathi opens conversation with a context-aware opener that references something specific from the day (not generic)
 - [] Sarathi surfaces at least 3 of today's captures during the conversation
 - [] Sarathi references morning goals by name and asks about each
 - [] Response streamed via WebSocket — no waiting for full response to load
 - [] User can respond via voice (Chirp) or text
 - [] After debrief, Sarathi generates an evening work plan with specific tasks and time estimates
 - [] Full conversation persisted in Memorystore Redis (active) + Cloud SQL (long-term)
 - [] Conversation turns embedded and added to Vertex AI Matching Engine
 - [] Debrief completable in 5–10 minutes if user is engaged
 - [] Lifecycle phase updated to evening in Memorystore on trigger
-

FEATURE 07 — Laptop Screen Monitoring (Desktop Web)

Description: Desktop web app monitors screen every 30 seconds. In MVP, frames sent to Gemini 2.5 Flash-Lite for classification (local Ollama added in Phase 2). Distractions flagged with goal-tied nudge. Session data synced to Cloud SQL as habit summaries.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Flash-Lite, Cloud SQL, Vertex AI Matching Engine, text-embedding-004, Firebase Authentication Web SDK

Acceptance Criteria:

- [] User opens Sarathi web app and authenticates via Firebase Authentication
 - [] User starts a focus session — app requests screen share permission (browser API)
 - [] Screen frame captured every 30 seconds as compressed JPEG (base64, quality 0.4)
 - [] Frame sent to Gemini 2.5 Flash-Lite with classification prompt
 - [] Response: PRODUCTIVE or DISTRACTION (one word)
 - [] DISTRACTION for 5+ consecutive minutes triggers nudge overlay on screen
 - [] Nudge message generated by Gemini referencing user's current active goal by name
 - [] Nudge is dismissable — logs distraction event regardless
 - [] Focus session summary shown at end: total focus time, distraction count, longest streak
 - [] Session summary POSTed to FastAPI as habit_summary — stored in Cloud SQL
 - [] Summary embedded and added to Vertex AI Matching Engine
 - [] Raw frames never stored or sent to any database
 - [] Screen monitoring toggleable on/off in settings
-

FEATURE 08 — Night Debrief & Alarm Setup

Description: Before sleep, Sarathi initiates a night debrief. Gemini 2.5 Pro reflects on the full day using RAG context. Sarathi identifies improvement areas, then sets the next morning's alarm via AlarmManager.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Pro, Vertex AI Matching Engine, Memorystore Redis, Cloud SQL, Firebase Cloud Messaging

Acceptance Criteria:

- [] App detects likely sleep time based on schedule config + time patterns in Redis
 - [] FCM push notification sent: "Ready to wrap up the day?"
 - [] Night debrief conversation covers: achievements today, what went wrong, one specific improvement area
 - [] Sarathi's feedback references actual events — captures, focus session data, goal check-ins
 - [] Feedback is honest and specific — not generic or pampering
 - [] After debrief, Sarathi asks: "What time do you want to wake up tomorrow?"
 - [] Sarathi suggests optimal wake time based on sleep goal config
 - [] User confirms or adjusts the time
 - [] Alarm set in Android system via AlarmManager
 - [] Alarm time stored in Cloud SQL alarms table
 - [] Full day summary stored in Cloud SQL as a conversation_turn with role=system
 - [] Summary embedded and added to Vertex AI Matching Engine for long-term memory
 - [] Lifecycle phase reset to morning in Memorystore after alarm is set
 - [] Night debrief completable in under 10 minutes
-

FEATURE 09 — General Conversational Chat

Description: At any point, user can open Sarathi and have a natural conversation. Gemini 2.5 Pro uses full RAG context. User can update goals or captures by talking naturally.

Google Cloud Services: Cloud Run (FastAPI), Vertex AI Gemini 2.5 Pro, Vertex AI Matching Engine, Memorystore Redis, Cloud SQL

Acceptance Criteria:

- [] Chat accessible from main navigation at all times
 - [] All messages processed by Gemini 2.5 Pro with Matching Engine RAG context
 - [] Sarathi knows user's goals, schedule, recent captures, and habit patterns in every response
 - [] User can update a goal by talking naturally: "Add a new goal: finish chapter 5 tonight" — Sarathi extracts and saves to Cloud SQL
 - [] User can add a capture by talking: "Remember I need to submit the assignment by Friday" — saved as capture in Cloud SQL
 - [] Conversation history persists across app restarts (Cloud SQL)
 - [] Streaming response — first tokens appear within 2 seconds
 - [] Response latency under 3 seconds for typical messages on good internet
-

FEATURE 10 — Settings & Profile Management

Description: User can view and update all profile data, goals, schedule, and app preferences. All changes immediately re-embedded and updated in Vertex AI Matching Engine.

Google Cloud Services: Cloud Run (FastAPI), Cloud SQL, Vertex AI Matching Engine, text-embedding-004, Google Maps Places API

Acceptance Criteria:

- [] User can view and edit all goals — changes synced to Cloud SQL + re-embedded in Matching Engine
- [] User can view and edit daily schedule
- [] User can update home location using Google Places API autocomplete
- [] User can toggle screen monitoring on/off (desktop web setting)
- [] User can view all today's captures with resolved/unresolved status
- [] User can clear conversation history (Cloud SQL + Matching Engine + Memorystore)
- [] User can view their current lifecycle phase
- [] Settings screen loads in under 2 seconds
- [] All updates immediately reflected in RAG memory

4. Sprint Plan

> Sprint duration: **1 week each**

> Total MVP sprints: **8 sprints (~8 weeks)**

Pre-Sprint 0 — Google Cloud Setup (2–3 days, before coding)

Goal: All GCP services provisioned and accessible before Sprint 1 begins.

Task	GCP Service	Notes
Create GCP project <code>sarathi-prod</code>	GCP Console	Enable billing + credits
Enable all required APIs	API Library	See Section 2.2 checklist
Create Firebase project	Firebase Console	Link to GCP project
Enable Firebase Auth (email/password)	Firebase Console	
Enable Firebase Cloud Messaging	Firebase Console	
Create Cloud SQL instance	Cloud SQL	PostgreSQL 15, db-f1-micro, us-central1
Create Memorystore Redis	Memorystore	1GB, Basic tier, us-central1
Create GCS bucket <code>sarathi-app</code>	Cloud Storage	Regional, us-central1

Task	GCP Service	Notes
Create Vertex AI Matching Engine index	Vertex AI	768-dim, cosine similarity
Get Google Maps API key	Maps Platform	Enable Geofencing, Routes, Places APIs
Store all secrets in Secret Manager	Secret Manager	DB password, Maps key, etc.
Create Cloud Run service account	IAM	Roles: Cloud SQL Client, Vertex AI User, Storage Object Admin, Secret Accessor
Set up GitHub repo + Cloud Build trigger	Cloud Build	Trigger on push to main

Sprint 1 — Backend Foundation + Cloud SQL + Firebase Auth

Goal: Working FastAPI backend on Cloud Run with auth, database schema, and Vertex AI RAG pipeline.

Task	Service	Estimate
FastAPI project setup + Dockerfile	Cloud Run	0.5 day
Firebase Admin SDK integration (auth verification)	Firebase Auth	0.5 day
Cloud SQL connection + all table creation	Cloud SQL	0.5 day
Memorystore Redis connection + session cache	Memorystore	0.5 day
Goals API — CRUD endpoints	Cloud Run + Cloud SQL	0.5 day
Schedule API — GET/PUT endpoints	Cloud Run + Cloud SQL	0.5 day
Vertex AI Matching Engine — connect + embed + store	Vertex AI	1 day
Basic RAG — retrieve_memory() function	Vertex AI + text-embedding-004	0.5 day
Deploy to Cloud Run via Cloud Build	Cloud Build + Artifact Registry	0.5 day

Sprint 1 Done When: Can register a user via Firebase Auth, save goals to Cloud SQL, embed them into Vertex AI Matching Engine, and retrieve relevant memories via RAG query. Cloud Run endpoint returns 200.

Sprint 2 — Gemini Integration + Chat API

Goal: Working Sarathi conversation with full RAG memory context via Gemini 2.5 Pro.

Task	Service	Estimate
Vertex AI Gemini client setup	Vertex AI	0.5 day
Prompt builder — system prompt + RAG + history	Vertex AI + Memorystore	1 day
Chat streaming via WebSocket	Cloud Run + WebSocket	1 day
Conversation history — Redis cache + Cloud SQL persist	Memorystore + Cloud SQL	0.5 day
Lifecycle phase tracking in Redis	Memorystore	0.5 day
React web app scaffolding + chat screen	Firebase Hosting + React	1 day
Firebase Auth Web SDK integration	Firebase Auth	0.5 day

Sprint 2 Done When: Can have a real Sarathi conversation via the web app. Gemini references your actual goals from Matching Engine memory. Responses stream in real time.

Sprint 3 — Morning Flow (Android)

Goal: Morning briefing + wake-up flow working end to end on Android.

Task	Service	Estimate
Android project setup — Kotlin + Compose + Hilt	Android	0.5 day
Firebase Auth SDK + login screen	Firebase Auth Android	0.5 day
Firebase Cloud Messaging setup	FCM Android	0.5 day
WorkManager alarm check job	Android WorkManager	0.5 day
AlarmManager exact alarm setup	Android AlarmManager	0.5 day
Wake-up screen UI — Gemini-generated motivation	Compose + Vertex AI	1 day
Morning briefing API — GET /lifecycle/morning	Cloud Run + Gemini	0.5 day
Google Maps Routes API — time-to-leave	Maps Platform Android	0.5 day

Task	Service	Estimate
Briefing screen UI + countdown timer	Compose	1 day

Sprint 3 Done When: Alarm fires on Android, wake-up screen shows Gemini-generated motivation referencing your actual goals, morning briefing loads with Routes API travel time.

Sprint 4 — Quick Capture (Android)

Goal: All three capture modes — voice, text, image — working and syncing to Cloud SQL and Matching Engine.

Task	Service	Estimate
Captures API — POST /captures	Cloud Run + Cloud SQL	0.5 day
Text capture UI	Compose	0.5 day
Voice capture — Chirp integration	Cloud Speech-to-Text	1 day
Image capture — CameraX + GCS upload	CameraX + Cloud Storage	1.5 days
GCS signed URL generation for images	Cloud Storage	0.5 day
Gemini Flash-Lite auto-categorization	Vertex AI Flash-Lite	0.5 day
Embed capture + store in Matching Engine	Vertex AI	0.5 day
Captures list screen — view + resolve	Compose + Cloud SQL	0.5 day
Floating action button (persistent)	Compose	0.5 day

Sprint 4 Done When: Can capture a voice note (Chirp transcribes accurately), a photo (uploaded to GCS), and a text note while at college. All three appear in captures list with correct auto-category.

Sprint 5 — Homecoming Detection & Evening Debrief

Goal: Location-triggered evening debrief working end to end on Android.

Task	Service	Estimate
Google Maps Geofencing API setup	Maps Platform Android	1 day

Task	Service	Estimate
Home location save — Places API autocomplete	Maps Platform Android	0.5 day
Geofence ENTER → FCM trigger → POST /lifecycle/homecoming	FCM + Cloud Run	1 day
Evening debrief Gemini prompt — surfaces captures + goals	Vertex AI Gemini + Matching Engine	1 day
Debrief chat screen UI	Compose	0.5 day
WebSocket streaming on Android	OkHttp WebSocket	0.5 day
Evening work plan generation after debrief	Vertex AI Gemini	0.5 day
Manual homecoming trigger in Settings	Compose + Cloud Run	0.5 day

Sprint 5 Done When: Walk into home area — get FCM notification — open debrief — Gemini references specific captures from the day and checks morning goals by name.

Sprint 6 — Laptop Screen Monitoring (Desktop Web)

Goal: Focus session with Gemini-powered screen monitoring working on desktop browser.

Task	Service	Estimate
Browser Screen Capture API integration	Web API	0.5 day
30-second frame capture + base64 compression	JavaScript	0.5 day
POST frame to Gemini 2.5 Flash-Lite for classification	Vertex AI Flash-Lite	1 day
PRODUCTIVE/DISTRACTION threshold logic (5 min)	Frontend	0.5 day
Nudge overlay UI — references user's current goal	React + Vertex AI	0.5 day
Focus session timer + stats display	React	0.5 day
POST /habits/summary after session	Cloud Run + Cloud SQL	0.5 day
Embed habit summary → Matching Engine	Vertex AI	0.5 day

Task	Service	Estimate
Session summary screen	React	0.5 day

Sprint 6 Done When: Start a focus session on desktop, open YouTube for 5+ minutes, Gemini-powered nudge appears on screen referencing your actual goal by name. Session summary shows correct focus time.

Sprint 7 — Night Debrief & Alarm Setup

Goal: Complete night loop — honest debrief, improvement insights, alarm set through conversation.

Task	Service	Estimate
Sleep time detection — schedule config + Redis time pattern	Memorystore + Cloud SQL	0.5 day
FCM trigger: POST /lifecycle/night → push notification	FCM + Cloud Run	0.5 day
Night debrief Gemini prompt — full day RAG context	Vertex AI Gemini + Matching Engine	1 day
Debrief chat UI (Android)	Compose	0.5 day
Alarm time extraction from conversation (Gemini structured output)	Vertex AI Gemini	0.5 day
AlarmManager: set alarm from Sarathi conversation	Android AlarmManager	0.5 day
POST /alarm — store in Cloud SQL	Cloud Run + Cloud SQL	0.5 day
Day summary embed → Matching Engine (long-term memory)	Vertex AI	0.5 day
Lifecycle phase reset to morning in Redis	Memorystore	0.5 day

Sprint 7 Done When: Night debrief runs — Gemini references specific things from the day — identifies one real improvement area — alarm set through conversation — fires correctly next morning.

Sprint 8 — Polish, Settings & End-to-End Testing

Goal: Full lifecycle works without breaking. All GCP service errors handled gracefully. Smooth enough to use daily for 2 weeks.

Task	Notes	Estimate
Settings screen Android — goals, schedule, location, prefs	Full CRUD via Cloud SQL	1 day
Settings screen Web — same + screen monitoring toggle	React	0.5 day
GCP error handling — Cloud SQL down, Vertex AI timeout, FCM failure	Graceful fallbacks	1 day
Edge case testing — no internet, empty captures, missed geofence	Test on real device	1 day
Cloud Monitoring alerts — set up alerts for errors + latency	Cloud Monitoring	0.5 day
Firebase Crashlytics integration	Firebase Crashlytics	0.5 day
Full lifecycle run — morning → day → evening → night	End-to-end test	0.5 day
Performance — Cloud Run cold start, Matching Engine latency	Optimize if needed	0.5 day

Sprint 8 Done When: Complete a full simulated day without any blocking bugs. All 10 feature acceptance criteria pass. Cloud Monitoring dashboard is live.

5. Definition of Done

Feature Level

A feature is DONE when:

- ☐ All acceptance criteria in Section 3 are passing
- ☐ FastAPI endpoints tested via Postman or Thunder Client
- ☐ Android screens tested on a real physical device
- ☐ No app crashes on the happy path
- ☐ Data correctly stored in Cloud SQL and Vertex AI Matching Engine
- ☐ No API keys or secrets hardcoded — all via Secret Manager
- ☐ Code committed and deployed to Cloud Run via Cloud Build

MVP Level

The MVP is DONE when:

- ☐ All 10 features are at feature-level DONE
- ☐ Creator has used Sarathi for 7 consecutive days as their daily system

- [] Creator rates daily usefulness 7/10 or above after those 7 days
- [] All infra costs billing to Google Cloud credits — zero out-of-pocket
- [] Cloud Monitoring dashboard shows no critical errors
- [] Firebase Crashlytics shows zero unresolved crashes

6. Out of Scope for MVP

Feature	Reason Deferred
Local Gemma model (Sarathi-Mini / Max)	Adds weeks of complexity; Gemini API sufficient to validate concept
Ollama / MediaPipe integration	Phase 2 — after concept is validated
Fine-tuning pipeline	Needs MVP data to train on; Phase 3
Multi-user backend	Single user to start; scale after validation
Native desktop Electron app	Web interface sufficient for MVP
College system integrations	Removed from product scope
Gemini Live API voice sessions	Nice-to-have; standard text chat first
Cloud Text-to-Speech (Sarathi voice)	Optional; text responses first
Analytics dashboard	Phase 4
Multi-language UI	Hindi/English both handled naturally by Gemini
AlloyDB	Cloud SQL sufficient for MVP scale

7. Risk Register

Risk	Probability	Impact	Mitigation
Vertex AI Gemini quota limits during testing	Medium	High	Request quota increase early; use Flash-Lite for non-critical tasks
Matching Engine cold start latency (first query is slow)	High	Medium	Warm up index on app start; cache frequent queries in Redis
Cloud SQL connection limits on Cloud Run	Medium	Medium	Use Cloud SQL connector with connection pooling (pg8000 or SQLAlchemy pool)

Risk	Probability	Impact	Mitigation
Android Geofence doesn't fire reliably indoors	High	Medium	Manual fallback trigger; lower threshold; test on multiple devices
FCM delivery delay on certain Android OEMs (Xiaomi, Realme)	High	Medium	Test on target device; use high-priority FCM messages
Screen Capture API blocked by browser security	Low	High	Test on Chrome first; document Electron fallback for Phase 2
Gemini frame classification is slow (screen monitoring)	Medium	Medium	Reduce frame quality further; batch if needed; Flash-Lite is fast
GCP credits run out during heavy development	Low	High	Monitor billing dashboard weekly; set budget alert at 50%
Google Maps Geofencing API billing spike	Low	Medium	Geofence ENTER only — very few API calls; Maps free tier covers it
Scope creep delays MVP	High	High	Hard feature freeze after Sprint 0; no new features until MVP ships
Cloud Run cold starts add latency to first request	Medium	Low	Set minimum instances to 1 for dev; acceptable for MVP

Sarathi MVP Document v2.0 | April 2026 | Google Cloud Edition | Confidential